

CHAPITRE V Les Sous Programmes

I Introduction

Lors de l'analyse de résolution d'un problème, il s'avère parfois nécessaire de décomposer ou de diviser le problème initial en sous-problèmes de moindre difficulté. Les sous problèmes peuvent à leur tour être décomposés en sous sous-problèmes.

On peut aussi être amené parfois à utiliser une portion du programme ou de l'algorithme plus d'une fois pendant l'exécution et vouloir éviter de réécrire ces instructions chaque fois qu'on en a besoin.

Les blocs d'instructions ainsi définis peuvent être regroupés en des modules qui ne font pas parti des instructions de l'algorithme principal mais auxquels on fait référence par le biais d'un "appel".

On distingue deux types de sous-programmes : les procédures et les fonctions.

I.1 Les Procédures

Définition

Une procédure est un sous-programme qui permet lors de son utilisation (appel) d'effectuer un certain nombre de traitements.

Vocabulaire et Syntaxe

Syntaxe

```
Nom_procédure ( [param1: type1, param2 : type2 ,...])
Var ....      [ Déclaration des variables locales à la procédure]
Début
    Inst 1
    Inst 2
    .
    .
    Inst n
Fin Nom_procédure
```

Appel

L'appel de la procédure se fait sous la forme d'une instruction de l'algorithme :

```
Algorithme nom_algo
Var
....
Début
    Instruction 1
    Instruction 2
    .
    .
    Nom_procédure ( [ para1, para2, ... ] )
    .
    Instruction n
Fin nom_algo
```

Remarques

- Les paramètres (noms de variables) qui se trouvent dans l'entête de déclaration de la procédure sont appelés paramètres fictifs (formels). Ils seront remplacés lors de l'appel par les paramètres effectifs.
- Lors de l'appel il faut respecter le nombre, l'ordre et le type de chaque paramètre formel.
- Si un paramètre effectif est appelé à voir sa valeur modifiée par les instructions de la procédure, il faudra le préciser dans l'entête de déclaration de la procédure en faisant précéder le paramètre formel par le mot "var". ce type de passage est dit : "passage par variable".
- Les variables déclarées dans le corps de la procédure sont inconnues (invisibles) par l'algorithme principal ou le module appelant (une procédure peut faire appel à une autre procédure)

Activité 1 : passage de paramètres par valeur

Ecrire un algorithme qui permet de saisir un tableau T de n entiers, $n \geq 40$, puis de mettre les éléments négatifs dans le tableau Tneg et les éléments positifs dans Tpos. Afficher les deux tableaux.

Correction

Procédure affich_tab (Tab : tableau de 40 entiers, nelem : entier)

Var cp : entier

début

Pour cp de 1 à nelem Faire
 écrire (Tab [cp] , " ")

Finpour

Fin affich_tab

Algorithme éclater_tableau

Var

T, Tpos, Tneg : Tableau de 40 entiers

n, i, j, k : entier

Début // de l'algorithme principal

Répéter

 écrire ("donnez la taille du tableau ≤ 40 ")

 lire (n)

 jusqu'à ((n>0) et (n $\leq$ 40))

 pour i de 1 à n Faire

 écrire ("donnez un élément du tableau :")

 lire (T [i])

 Finpour

 j $\leftarrow$ 0 k $\leftarrow$ 0

 Pour i de 1 à n Faire

 Si (T[i] $\geq$ 0) alors

 j $\leftarrow$ j+1

 Tpos [j] $\leftarrow$ T[i]

 Sinon

 k $\leftarrow$ k + 1

 Tneg [k] $\leftarrow$ T[i]

 Finsi

 Finpour

 affich_tab(Tpos, j)

 affich_tab (Tneg, k)

Fin éclater_tableau

Activité 2 : passage de paramètres par variable

Ecrire un algorithme qui permet de saisir deux tableaux T1 et T2 d'entiers, de tailles respectives n et m, ($n \leq 40$ et $m \leq 40$), puis de calculer la somme des produits des éléments de T1 par chacun des éléments de T2 (ou chaque élément de T2 par chaque élément de T1)

Par exemple si l'on a :

T1	4	8	7	12
T2	3	6		

$$\text{somprod} = 3 * 4 + 3 * 8 + 3 * 7 + 3 * 12 + 6 * 4 + 6 * 8 + 6 * 7 + 6 * 12 = 279$$

Correction

Lors de l'analyse, on remarque deux traitements qui sont répétés : la saisie de la taille et le remplissage de chacun des tableaux T1 et T2. On va écrire la procédure saisir_n et la procédure remplir_tab.

Procédure saisir_n (var ne : entier)

début

répéter

 écrire ("donnez la taille du tableau ")

 lire (ne)

jusqu'à ((ne>0) et (ne <=40))

Fin saisir_n

Procédure remplir_tab (var t : tableau de 40 entier, var nelem : entier)

var cp : entier

Début

 saisir_n(nelem)

pour cp de 1 à nelem faire

 écrire ("donnez un élément du tableau :")

 lire (t[cp])

finpour

Fin remplir_tab

Algorithme som_Prod_tableaux

Type

 TabEnt = tableau [1..40] d'entier

Var

 T1, T2: TabEnt

 sp, n1, n2 : entier

Début

 remplir_tab (T1, n1)

 remplir_tab (T2, n2)

 sp $\leftarrow$ 0

pour i de 1 à n1 faire

pour j de 1 à n2 Faire

 sp $\leftarrow$ sp + T1[i] * T2 [j]

finpour

finpour

 écrire ("somme des produits = ", sp)

Fin som_prod_tableaux

I.2 Les Fonctions

Définition

Une fonction est un sous-programme qui permet lors de son utilisation (appel) d'effectuer un certain nombre de traitements puis de renvoyer un résultat unique de type simple.

Syntaxe

```
Nom_fonction ( [param1: type1, param2 : type2 ,...] ) : Type_résultat
Var .....    [ déclaration des variables locales à la fonction ]
Début
    Inst 1
    Inst 2
    ...
    Inst n
    retourne (resultat)
Fin Nom_fonction
```

Appel

L'appel à une fonction peut se faire de deux manières :

- Sous la forme d'une instruction dans le module appelant
 - en faisant apparaître le nom de la fonction dans une expression ou une instruction d'affectation ou d'affichage du résultat parmi les instructions du module appelant..
- Dans le cas d'un appel par affectation, le type de la variable doit être compatible avec le type de retour (type_résultat) de la fonction

1 ^{ère} Manière : appel par affectation du résultat de la fonction à une variable	2 ^{ème} Manière : instruction d'appel sans récupération du résultat de la fonction
Algorithme nom_algo Début Instruction 1 Instruction 2 ... v ← Nom_fonction ([para1, para2, ...]) ... Instruction n Fin nom_algo	Algorithme nom_algo Début Instruction 1 Instruction 2 ... Nom_fonction ([para1, para2, ...]) ... Instruction n Fin nom_algo

Remarques

- les paramètres formels et les paramètres effectifs doivent correspondre de point de vue nombre, type et ordre
- les paramètres formels d'une fonction ont un mode de passage par valeur ou par variable

Activité

Modifier l'algorithme précédent en remplaçant la procédure saisir_n par une fonction.

